جامعة الوادي
كلية العلوم الدقيقة
قسم الإعلام الآلي

TP04: Containerization and Big Data Environment Deployment Using Docker

إعداد الطالبة: آمنة كنيوة

السنة الجامعية 2026--2027

١. مقدمة

أصبح التعامل مع البيانات وتحليلها يتطلب استخدام أدوات وتقنيات حديثة تساعد على تنظيم بيئة العمل وتسهيل تشغيل التطبيقات. ومن بين هذه التقنيات نجد ،Docker الذي يسمح بتشغيل التطبيقات داخل بيئات معزولة تسمى الحاويات.

في هذا العمل التطبيقي، قمنا بإنشاء تطبيق باستخدام لغة Python ومكتبة Pandas لتحليل مجموعة بيانات PlantVillage الخاصة بصور أوراق الطماطم وأمراضها. بعد ذلك، استخدمنا Docker لتجهيز بيئة التشغيل وإنشاء صورة للتطبيق وتشغيله داخل الحاويات.

كما استخدمنا Hub Docker لمشاركة صورة التطبيق، وCompose Docker لتنظيم عملية تشغيله.

٢. أهداف العمل

يهدف هذا العمل التطبيقي إلى:

- التعرف على مفهوم Docker وطريقة عمله.
- إنشاء ملف Dockerfile لتجهيز بيئة التطبيق.
- إنشاء صورة Docker انطلاقًا من Dockerfile.
- تشغيل التطبيق داخل حاوية Docker.
- ربط ملفات البيانات بالحاوية باستخدام Volumes.
- إنشاء شبكة Network. Docker
- استخدام Hub Docker لتخزين ومشاركة الصور.
- استخدام Compose Docker لتسهيل إدارة التطبيق.
- إجراء تحليل أولي للبيانات باستخدام Python وPandas.

٣. الأدوات والتقنيات المستعملة

الأداة	وظيفتها
Docker	إنشاء وتشغيل الحاويات.
Dockerfile	تحديد تعليمات تجهيز بيئة التطبيق.
Compose Docker	تنظيم عملية تشغيل التطبيق.
Hub Docker	تخزين ومشاركة صور Docker.
Python	برمجة تطبيق تحليل البيانات.
Pandas	قراءة ومعالجة ملفات CSV.
PlantVillage	مجموعة البيانات المستعملة.
Terminal	تنفيذ أوامر Docker.

جدول ١.٣: الأدوات والتقنيات المستعملة

٠٤ وصف مجموعة البيانات

استعملنا مجموعة بيانات PlantVillage الخاصة بأوراق النباتات، وركزنا على صور أوراق الطماطم.

تحتوي المجموعة على صور لأوراق سليمة وأخرى مصابة بأمراض مختلفة. قمنا بتجهيز البيانات في ملف CSV يحمل الاسم:

`PlantVillage_PowerBI.csv`

يحتوي الملف على خمسة أعمدة أساسية:

اسم العمود	الوصف
Image_Name	اسم الصورة.
Plant_Class	فئة النبات.
Plant_Name	اسم النبات.
Disease	المرض.
Image_Path	مسار الصورة.

جدول ١٠٤: أعمدة مجموعة البيانات

٥. تنظيم ملفات المشروع

تم تنظيم المشروع داخل مجلد باسم TP04_Docker كما يلي:

```
TP04_Docker/
|
|-- Dockerfile
|-- app.py
|-- create_dataset.py
|-- compose.yaml
|
|-- Tomato/
|
|-- data/
    |-- PlantVillage_PowerBI.csv
```

١.٥ البرنامج الرئيسي app.py

يمثل هذا الملف البرنامج المسؤول عن تحليل البيانات. استعملنا لغة Python ومكتبة Pandas لقراءة ملف CSV، ثم عرض عدد الصفوف والأعمدة وأسماء الأعمدة وأول خمسة سجلات.

```
import os
import pandas as pd

DATA_PATH = "/app/data/PlantVillage_PowerBI.csv"

print("=== TP04 Docker Data Analysis ===")

if not os.path.exists(DATA_PATH):
    raise FileNotFoundError(
        f"Dataset not found: {DATA_PATH}"
    )
```

```
df = pd.read_csv(DATA_PATH)

print(f"Dataset: {os.path.basename(DATA_PATH)}")
print(f"Number of rows: {len(df)}")
print(f"Number of columns: {len(df.columns)}")

print("\nColumn names:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
print(df.head().to_string(index=False))

print("\nAnalysis completed successfully!")
```

٢.٥ ملف Dockerfile

يحتوي هذا الملف على التعليمات اللازمة لإنشاء بيئة تشغيل التطبيق داخل Docker.

```
FROM python:3.12-slim

WORKDIR /app

COPY app.py .

RUN pip install --no-cache-dir pandas

CMD ["python", "app.py"]
```

شرح التعليمات:

- FROM: اختيار صورة أساسية تحتوي على Python 3.12.
- WORKDIR: تحديد مجلد العمل داخل الحاوية.
- COPY: نسخ ملف البرنامج.
- RUN: تثبيت مكتبة Pandas.
- CMD: تحديد أمر تشغيل التطبيق.

٣.٥ ملف compose.yaml

استعملنا Compose Docker لتنظيم إعدادات تشغيل التطبيق.

```
services:
  data-analysis:
    build: .
    container_name: tp04-analysis
    volumes:
      - ./data:/app/data:ro
    restart: "no"
```

شرح الإعدادات:

- services: تحديد الخدمات.
- data-analysis: اسم الخدمة.
- build: بناء الصورة اعتمادًا على Dockerfile.
- container_name: تحديد اسم الحاوية.
- volumes: ربط مجلد البيانات بالحاوية.
- ro: السماح بقراءة الملفات فقط.
- restart: عدم إعادة تشغيل الحاوية تلقائيًا.

٤.٥ الملف create_dataset.py

استعملنا هذا الملف في مرحلة إنشاء وتجهيز مجموعة البيانات، والنتيجة النهائية التي استخدمناها في التحليل هي ملف CSV الموجود داخل مجلد البيانات.

٥.٥ المجلدان Tomato و data

- Tomato: يحتوي على الصور الأصلية لأوراق الطماطم.
- data: يحتوي على ملف CSV المنظم المستعمل في التحليل.

٠٦ مراحل تنفيذ العمل التطبيقي

١٠٦ المرحلة الأولى: إنشاء مجلد المشروع

أنشأنا مجلدًا باسم TP04_Docker لتنظيم ملفات المشروع، ثم وضعنا ملفات Python وDocker والبيانات في أماكنها المناسبة.

٢٠٦ المرحلة الثانية: تجهيز مجموعة البيانات

جهزنا البيانات وحفظناها في الملف:

`data/PlantVillage_PowerBI.csv`

حتى يمكن استعمالها داخل تطبيق Python.

٣٠٦ المرحلة الثالثة: إنشاء Image Docker

استعملنا الأمر التالي لبناء صورة Docker:

```
docker build -t amina2003/tp04-analysis:1.0 .
```

يقوم الأمر بقراءة تعليمات Dockerfile وإنشاء صورة تحتوي على بيئة Python والتطبيق ومكتبة Pandas.

٤٠٦ المرحلة الرابعة: تشغيل التطبيق داخل Container

استعملنا الأمر التالي:

```
docker run --rm \
-v "$(pwd)/data:/app/data:ro" \
```

```
amina2003/tp04-analysis:1.0
```

يقوم هذا الأمر بتشغيل التطبيق داخل حاوية Docker.

- docker run: تشغيل حاوية جديدة.
- --rm: إزالة الحاوية بعد انتهاء التشغيل.
- -v: ربط مجلد البيانات من الجهاز بالمجلد داخل الحاوية.
- ro: جعل البيانات للقراءة فقط.

٥.٦ المرحلة الخامسة: رفع الصورة إلى Hub Docker

استعملنا Hub Docker لتخزين صورة التطبيق ومشاركتها.

أولًا قمنا بتسجيل الدخول:

```
docker login
```

ثم رفعنا الصورة:

```
docker push amina2003/tp04-analysis:1.0
```

ولتحميل الصورة استعملنا:

```
docker pull amina2003/tp04-analysis:1.0
```

نجحت عملية الرفع والتحميل، مما يسمح بإعادة استخدام صورة التطبيق على جهاز آخر يحتوي على Docker.

ملاحظة: الذي رفعناه إلى Hub Docker هو صورة Docker، وليس ملف CSV الموجود على جهازنا.

٦.٦ المرحلة السادسة: إنشاء Network Docker

أنشأنا شبكة Docker باسم:

```
docker network create tp04-network
```

تسمح الشبكة للحاويات المتصلة بها بالتواصل فيما بينها.

لمشاهدة الشبكات الموجودة:

```
docker network ls
```

ولمعرفة تفاصيل الشبكة:

```
docker network inspect tp04-network
```

بعد ذلك شغلنا عدة حاويات على الشبكة نفسها، باستعمال:

```
docker run \
--name tp04-node1 \
--network tp04-network \
-v "$(pwd)/data:/app/data:ro" \
amina2003/tp04-analysis:1.0
```

وكررنا التجربة مع الحاويتين tp04-node2 وtp04-node3.

انتهت الحاويات بحالة Exited (0)، وهذا يعني أن التطبيق أكمل تنفيذه بنجاح.

ملاحظة تقنية: تشغيل عدة حاويات من الصورة نفسها لا يعني تلقائيًا توزيع العمل بينها؛ في تجربتنا قمنا بتشغيل التطبيق نفسه في كل حاوية.

٧.٦ المرحلة السابعة: استخدام Compose Docker

لتشغيل التطبيق اعتمادًا على ملف compose.yaml استعملنا:

```
docker compose up --build
```

للتحقق من حالة الحاويات:

```
docker compose ps -a
```

ولإيقاف وإزالة الحاويات والشبكة الافتراضية الخاصة بالمشروع:

```
docker compose down
```

٧. النتائج المتحصل عليها

بعد تنفيذ التطبيق بنجاح، حصلنا على النتائج التالية:

المؤشر	النتيجة
عدد الصفوف	246,993
عدد الأعمدة	5
عدد السجلات المعروضة	5
حالة التنفيذ	ناجح

جدول ١٠٧: نتائج تحليل البيانات

أسماء الأعمدة التي ظهرت في النتائج:

```
Image_Name
Plant_Class
Plant_Name
Disease
Image_Path
```

كما تمكنا من:

١. إنشاء صورة Docker بنجاح.
٢. تشغيل التطبيق داخل حاوية Docker.
٣. ربط البيانات الموجودة على الجهاز بالحاوية.
٤. رفع الصورة إلى Hub Docker وتحميلها.
٥. إنشاء شبكة Docker وتشغيل عدة حاويات عليها.
٦. تشغيل التطبيق باستعمال Docker Compose.

٠٨ حدود العمل التطبيقي

ركز هذا العمل على تشغيل تطبيق تحليل بيانات بسيط داخل Docker، ولم يتضمن تدريب نموذج ذكاء اصطناعي أو بناء نظام موزع فعلي لتقسيم البيانات والمهام بين الحاويات.

كما أن إنشاء Network Docker لا يعني بحد ذاته أن الحاويات تتبادل البيانات؛ إذ يجب أن يتضمن التطبيق آلية اتصال وتبادل معلومات بينها.

٩. الخاتمة

من خلال هذا العمل التطبيقي، تعرفنا على تقنية Docker وطريقة استخدامها في تجهيز وتشغيل تطبيقات تحليل البيانات.

قمنا بإنشاء تطبيق باستخدام Python ومكتبة Pandas لتحليل مجموعة بيانات PlantVillage، ثم جهزنا بيئة تشغيله باستخدام Dockerfile وأنشأنا Image، Docker وبعد ذلك شغلنا التطبيق داخل Containers.

كما تعرفنا على Hub Docker لمشاركة الصور، وNetwork Docker لربط الحاويات، وDocker Compose لتنظيم عملية التشغيل.

سمح لنا هذا التطبيق بفهم المفاهيم الأساسية لتقنية Docker وتطبيقها عملياً في مشروع لتحليل البيانات.